

Attest.

A multimodal, self-grading RAG system for financial documents.

Ask real financial reports anything — and get an answer that **cites its sources, reads the charts** inside the document, and **grades its own honesty**, flagging itself whenever it cannot ground what it just said.

Author	Manglam Dubey
Date	July 2026
Status	Planning complete — ready to build

01 North-Star

What it is. Attest is a web application that lets a user ask plain-English questions across real financial reports (10-Ks, 10-Qs, annual reports) and returns answers that (a) cite their exact sources, (b) read the charts and figures inside the documents — not just the text — and (c) grade their own faithfulness, flagging any answer they cannot ground.

The problem it solves **Q2.** Analysts drown in hundreds of pages per filing. Keyword search misses context; ordinary RAG chatbots are blind to charts and hallucinate silently. Attest closes both gaps — **multimodal retrieval** for the charts, a **self-grading eval layer** for the honesty.

Target user **Q19.** Equity/credit analysts, finance students, and anyone who must pull grounded facts from long financial documents quickly. Secondary: it is Manglam's GenAI/RAG portfolio flagship.

Inspiration **Q18.** Not “to learn a skill.” The real problem: most RAG systems hand you a confident answer with no way to separate truth from hallucination — and go blind the moment the fact lives in a chart.

02 Success Criteria — frozen before building

These are the numbers set *before* a line is written. This is the honest answer to **Q3** — a criterion you can only claim if you set it up front.

Metric	Target (v1)	Measured how
Faithfulness	≥ 0.90	RAGAS, on the labeled eval set
Retrieval / context precision	≥ 0.85	RAGAS, against ground-truth sources
Answer relevance	≥ 0.85	RAGAS
Hallucination guard	$< 0.50 \rightarrow$ auto-flag	never shown as a confident answer
Figure-grounded questions	cite correct figure $\geq 80\%$	manual check on a chart-question set
Answer latency (p95)	≤ 8 s	engine timing logs

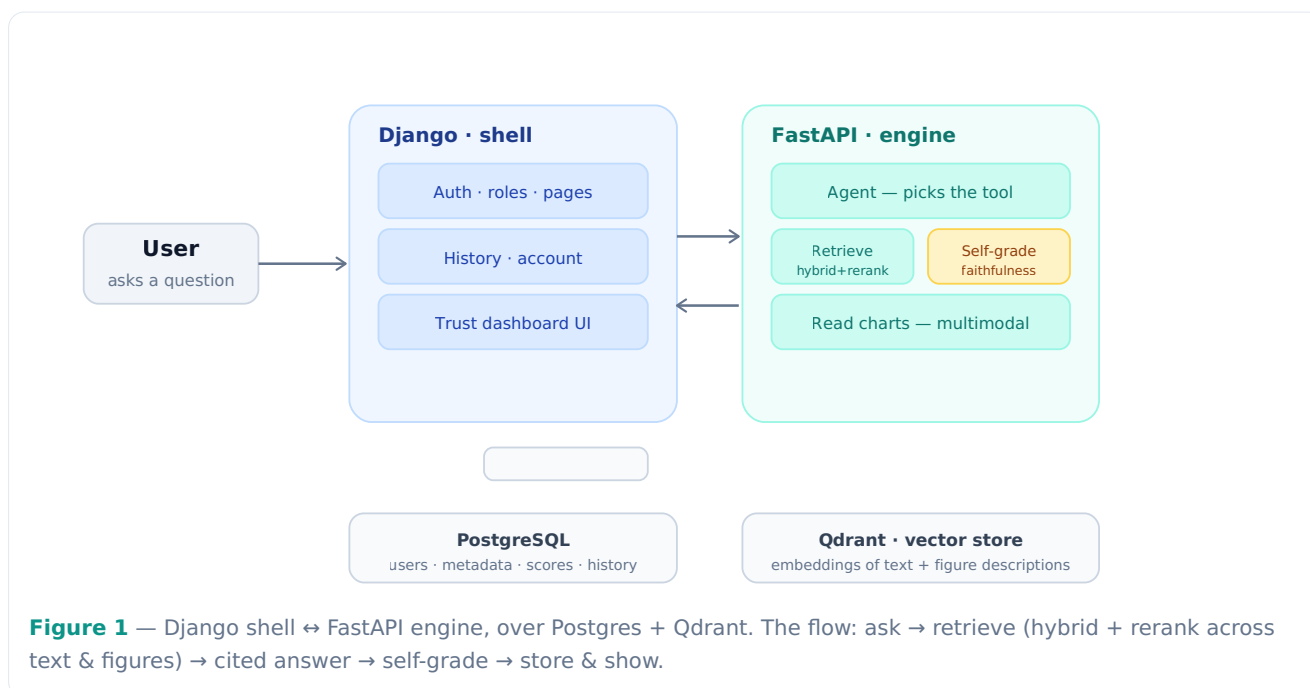
Push back on these. At interview *you* defend why faithfulness ≥ 0.90 and not 0.85. Treat this table as a contract you can renegotiate now — not after building.

03 The Corpus — where the documents come from

- **Live set** — ~10-20 real filings hand-collected from **SEC EDGAR** (10-K / 10-Q) plus glossy **annual reports** (annualreports.com / investor-relations pages). Self-collected on purpose: scraped data beats a clean Kaggle CSV as an initiative signal.
- **Labeled scoring set** — **FinQA** · **TAT-QA** · **ConvFinQA**: financial QA benchmarks built on real reports, each question shipping a human-verified answer. This becomes the eval harness's test set — the source of the \$02 numbers.
- **Why financial** — charts and tables are dense here, so the multimodal layer is genuinely *load-bearing*, not decorative.

04 Architecture Q16

Two clean halves: **Django** is the product shell (auth, roles, pages, history, the UI a user touches). It talks over HTTP to a **FastAPI** engine — the RAG + agent brain. **PostgreSQL** holds users, metadata, eval scores and history; **Qdrant** holds the vector embeddings of text *and* figure descriptions.



05 The Two Settled Forks — decided, with reasoning

Fork A — Vector store: **Qdrant** (over pgvector).

Qdrant is a resume-visible, purpose-built vector DB with first-class hybrid search, payload filtering and reranking hooks the project needs, and it keeps the vector concern cleanly separated from Postgres (a cleaner architecture story to defend). pgvector is simpler to run but hides the skill inside Postgres and is weaker at hybrid search. Cost: one extra container — Docker already handles it.

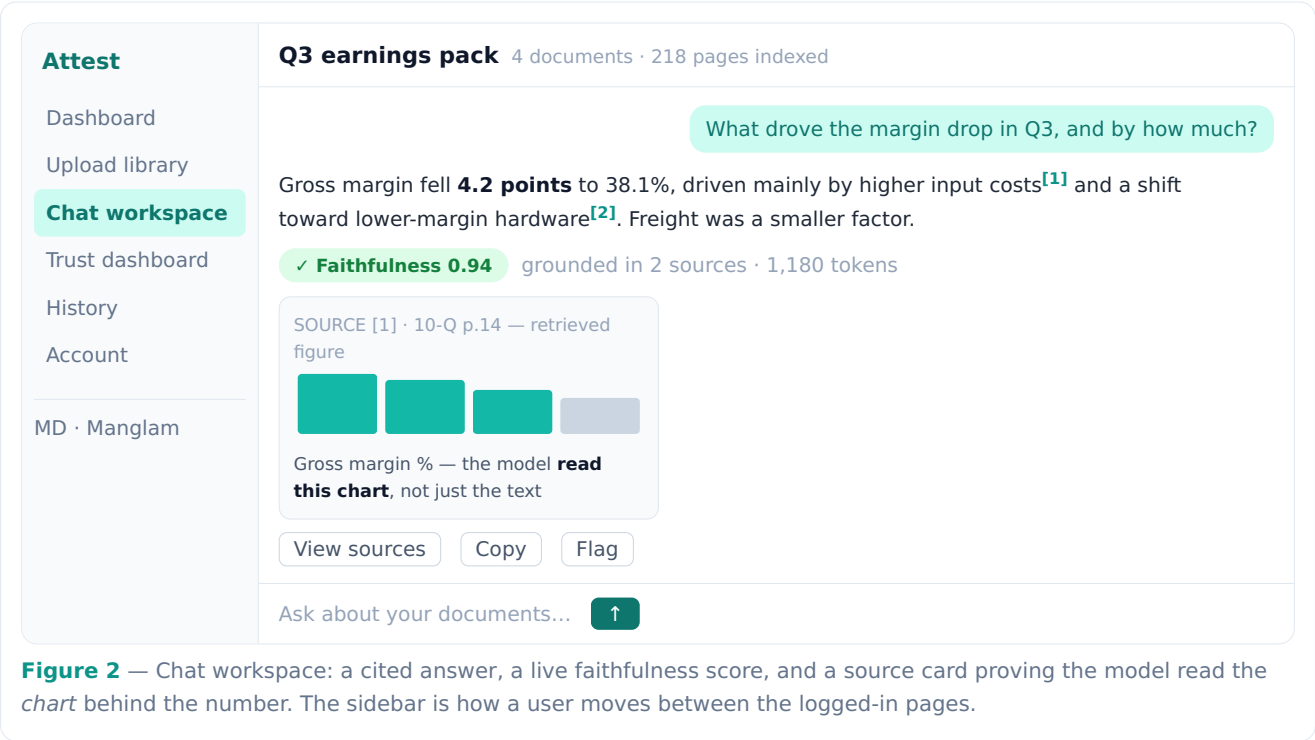
Fork B — Multimodal pattern: **image-to-text figure description** (over pure cross-modal embeddings).

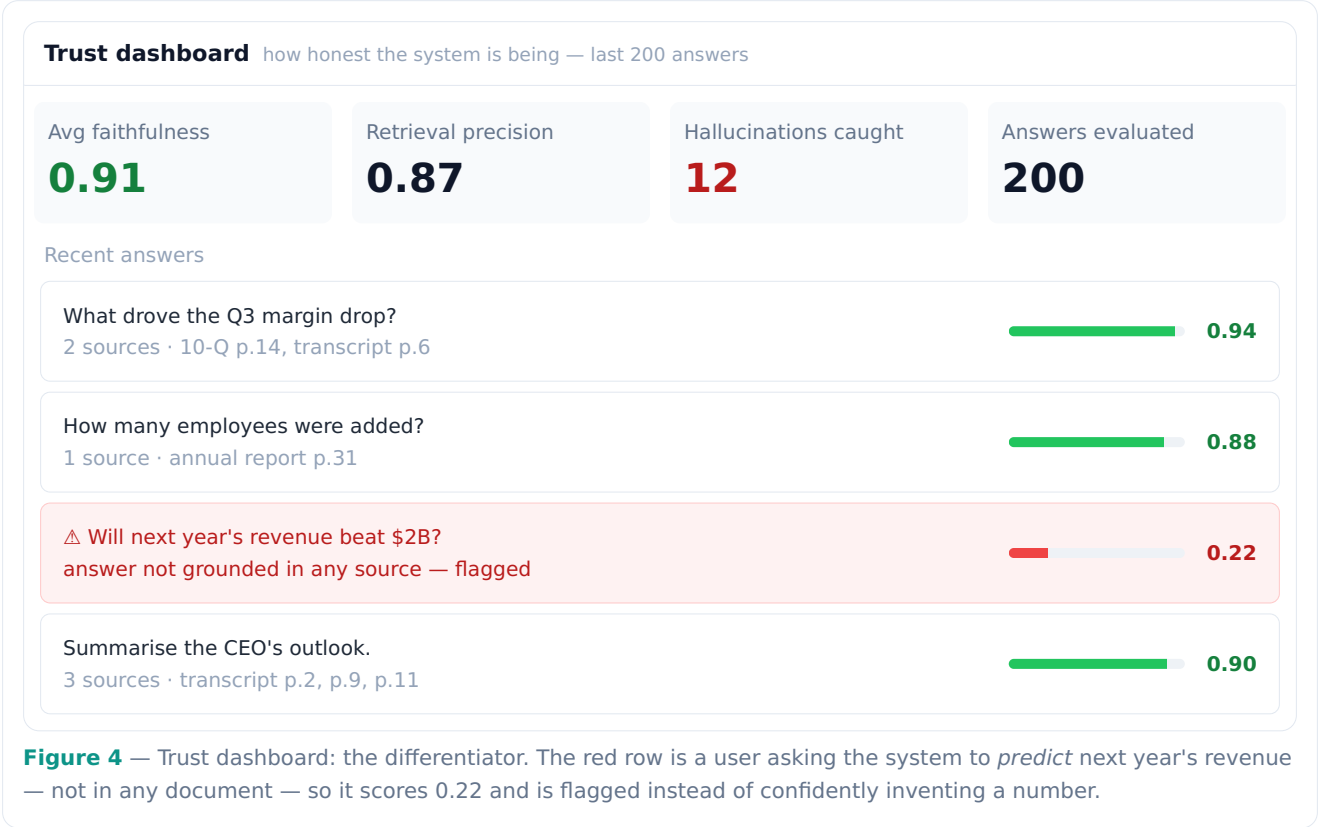
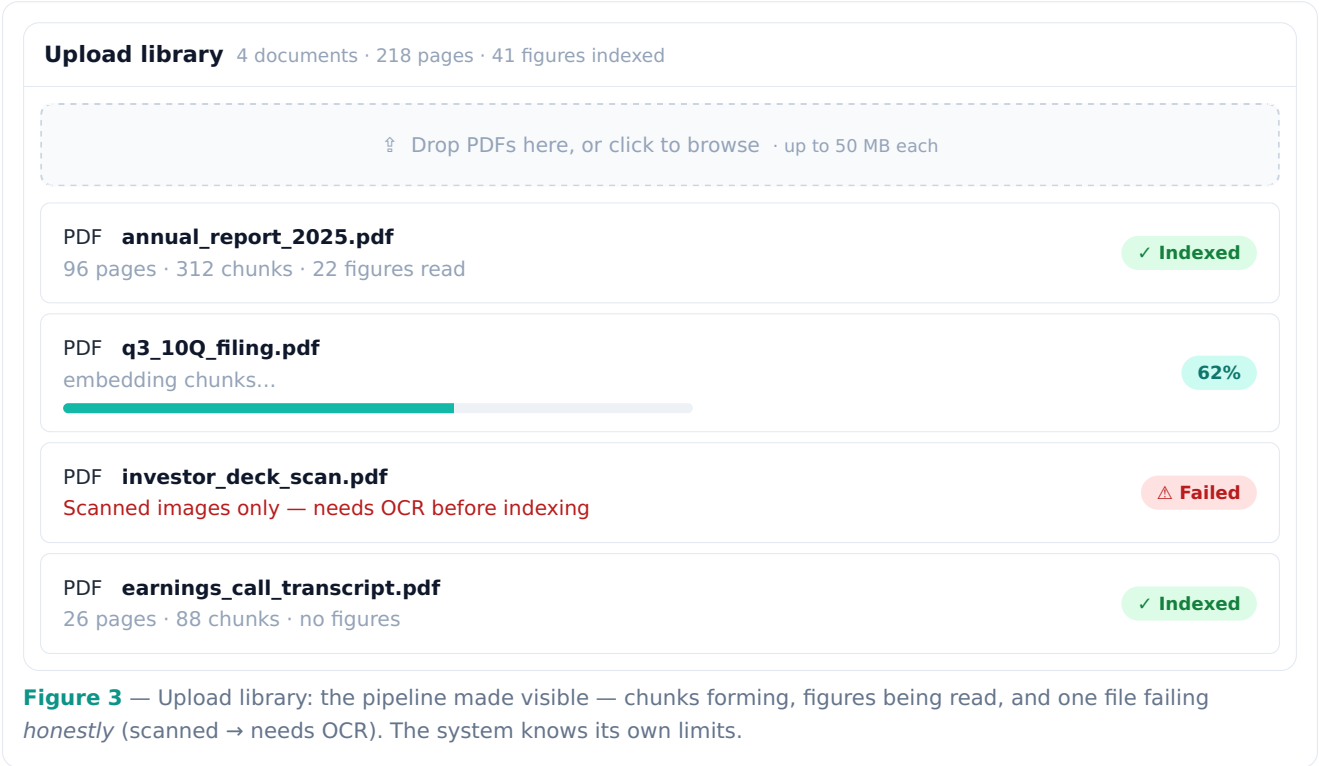
Each extracted figure is described by a vision-capable LLM into rich text, then embedded alongside the document text. This drops straight into the text-RAG pipeline from turns 1–2, is cheap, and is easy to evaluate (the description is human-inspectable). Pure CLIP-style embeddings are more elegant but harder to eval and integrate — parked as future work (\$10).

06 Page Map Q7 · Q19 · Q25

Zone	Pages
Public	Landing · Login · Signup
User (auth)	Dashboard · Upload / Library · Chat workspace · Answer & source viewer · Trust dashboard · History · Account
Admin (role-gated)	Ingestion monitor · Observability (token cost, health)

The user-vs-admin split is **real access control + multi-tenant isolation**: User A's documents never enter User B's retrieval. That is why auth is load-bearing here, not cosmetic.



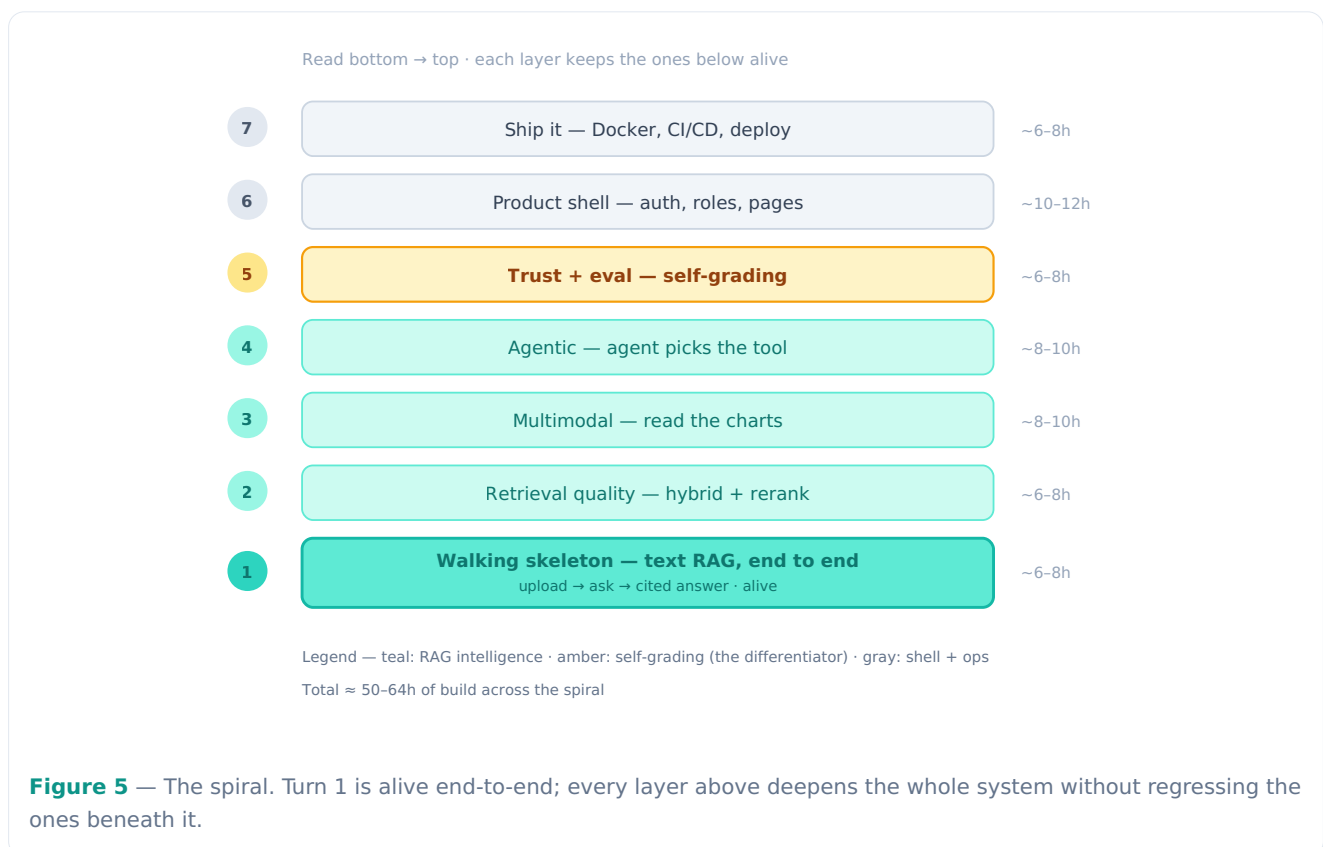


07 The Spiral, Turn by Turn Q11 · Q21

One product, born as a thin end-to-end skeleton, then each turn adds a capability layer *across the whole app* — never breaking what's below. Read the figure bottom-to-top.

#	Layer	"Done" means	Hours
---	-------	--------------	-------

1	Walking skeleton	a real question over one PDF returns a cited answer (text only, no auth)	~6-8
2	Retrieval quality	chunking + hybrid (vector + BM25) + cross-encoder rerank; precision measured on a small set	~6-8
3	Multimodal	figures extracted, described, embedded; a chart-only question answered correctly	~8-10
4	Agentic	LangGraph agent routes a multi-step question (retrieve / compare / summarise) with no hardcoding	~8-10
5	Trust + eval	RAGAS scores + the red-flag path work on an ungrounded question; §02 targets enforced	~6-8
6	Product shell	Django auth + roles + multipage; two users cannot see each other's corpus	~10-12
7	Ship it	Docker Compose + CI/CD + cost observability; a stranger can sign up and use it live	~6-8



08 HR-Question Capture — the 25, mapped

Every project-viva question is captured *by the build itself* or logged as it happens. Nothing is left to memory at interview time.

Turn / moment	Questions it locks in
1 · Skeleton	Q6 how it works · Q16 architecture

2 · Retrieval quality	Q4 ranked features · Q13/14 real challenge + fix
3 · Multimodal	Q2 problem via tech · Q20 proudest part
4 · Agentic	Q4 features · Q17 future work
5 · Trust + eval	Q3 success criterion · Q15 when it failed · Q20
6 · Product shell	Q7 / Q25 effort saved · Q19 target user
7 · Ship it	Q5 tools · Q11 phases + hours
Build journal (every turn)	Q13 · Q14 · Q15 — the real problem + concrete fix, written down when it happens
End-of-project	Q18 inspiration · Q21 prioritisation · Q23/24 feedback (a senior-review checkpoint)
Answered in one word	Q8-Q10 team — solo

The build journal is not optional. Q13/14/15 are exactly where a solo builder goes blank at interview. Log the real problem and its concrete fix each turn — while it's fresh.

09 Tech Stack — final Q5

Layer	Choice
Product shell	Django (auth, sessions, RBAC, ORM, templates) + HTMX for light interactivity — stays Python-side, no heavy JS
Engine	FastAPI
Orchestration	LangChain + LangGraph (agentic routing)
LLMs	a vision-capable model (e.g. Gemini) for figure description; a fast instruct model for generation
Embeddings	open sentence-embedding model (e.g. BGE) via sentence-transformers
Retrieval	hybrid (vector + BM25) + cross-encoder reranker
Vector store	Qdrant (Fork A)
Relational DB	PostgreSQL
PDF parsing	PyMuPDF (text + figure extraction) + Tesseract OCR fallback
Evaluation	RAGAS
Packaging / deploy	Docker Compose · GitHub Actions · Render (or similar)
Observability	token / cost tracking (+ optional MLflow for eval runs)

10 Out of Scope — v1 feeds Q17

Deliberate non-goals. Naming them now kills scope creep — and becomes an honest “future work” answer instead of a gap.

- Live market-data feeds / real-time prices
- User billing & payments
- Fine-tuning or training our own model
- Non-financial domains · multi-language documents
- Native mobile app · voice interface
- Pure cross-modal (CLIP) embeddings — deferred (see Fork B)

Frozen means frozen. The §02 numbers are the contract. Change them only with a reason you can defend at interview. Everything else grows one spiral layer at a time.